

LGD PD Modeling Analysis Report

Fan Peng

2025-07-13

目录

1	Data Loading	2
2	EDA	3
2.1	Exploratory data analysis	3
2.2	Visulization: lifetime value distributions and outliers	4
3	Correlation matrix	5
3.1	Spearman correlation	5
3.2	Spearman correlation heatmap	6
4	LGD(Loss Given Default)	7
4.1	Create target variable	7
4.2	Binary LGD label distribution	9
4.3	Feature overview	10
4.4	Split data into training and testing	10
4.5	Variable selection	12
4.6	Combination generating	12
4.7	binary LGD Logistic Regression	13
4.8	Continuous LGD Linear Regression	14
4.9	Save binary and continuous results	15
4.10	Logistic ROC curve	15
5	PD(Probability of Default)	18
5.1	Variable selection, combinations, and random forest tuning	18
5.2	Random forest feature importances	20
5.3	Random forest tuning results	22
6	Final output	22

1 Data Loading

```
# Import libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score

# Load the uploaded LGD dataset
lgd_df = pd.read_excel("Copy of lgd.xlsx")

# Show basic info
lgd_df.info()
```

```
## <class 'pandas.core.frame.DataFrame'>
## RangeIndex: 2545 entries, 0 to 2544
## Data columns (total 8 columns):
##  #   Column                Non-Null Count  Dtype
## ---  -
## 0   LTV                   2545 non-null   float64
## 1   Recovery_rate         2545 non-null   float64
## 2   lgd_time              2545 non-null   float64
## 3   y_logistic            2545 non-null   float64
## 4   lnrr                  2545 non-null   float64
## 5   Y_probit              2545 non-null   float64
## 6   purpose1              2545 non-null   int64
## 7   event                 2545 non-null   int64
## dtypes: float64(6), int64(2)
## memory usage: 159.2 KB
```

```
# Display first few rows
lgd_df.head()
```

```
##      LTV  Recovery_rate  lgd_time  ...  Y_probit  purpose1  event
## 0  0.214078      0.698016  0.301984  ... -0.518701         0       1
## 1  0.214078      0.780084  0.219916  ... -0.772477         0       1
## 2  0.214078      0.702287  0.297713  ... -0.530989         0       1
## 3  0.214078      0.753989  0.246011  ... -0.687096         0       1
```

```
## 4 0.214078      0.802813  0.197187  ... -0.851711      0      1
##
## [5 rows x 8 columns]
```

LGD Data Loading Summary This step imports Python libraries and loads the LGD dataset from “Copy of lgd.xlsx”. It shows that the dataset has 2,545 rows and 8 columns with no missing values. Main columns are LTV, Recovery_rate, lgd_time, y_logistic, Y_probit, purpose1, and event. The first rows confirm the data is clean and ready for analysis.

2 EDA

2.1 Exploratory data analysis

```
# Step 2: Quick EDA - summary stats and check for missing values
summary_stats = lgd_df.describe()
missing_values = lgd_df.isnull().sum()

summary_stats, missing_values
```

```
## (          LTV  Recovery_rate  ...      purpose1      event
## count  2545.000000    2545.000000  ...  2545.000000  2545.000000
## mean      0.676556      0.771870  ...    0.072692    0.713949
## std      0.364127      0.329109  ...    0.259681    0.452002
## min      0.001359      0.000010  ...    0.000000    0.000000
## 25%      0.399181      0.602146  ...    0.000000    0.000000
## 50%      0.659417      0.967934  ...    0.000000    1.000000
## 75%      0.923548      0.999990  ...    0.000000    1.000000
## max      1.984065      0.999990  ...    1.000000    1.000000
##
## [8 rows x 8 columns], LTV      0
## Recovery_rate      0
## lgd_time           0
## y_logistic         0
## lnrr               0
## Y_probit           0
## purpose1           0
## event              0
## dtype: int64)
```

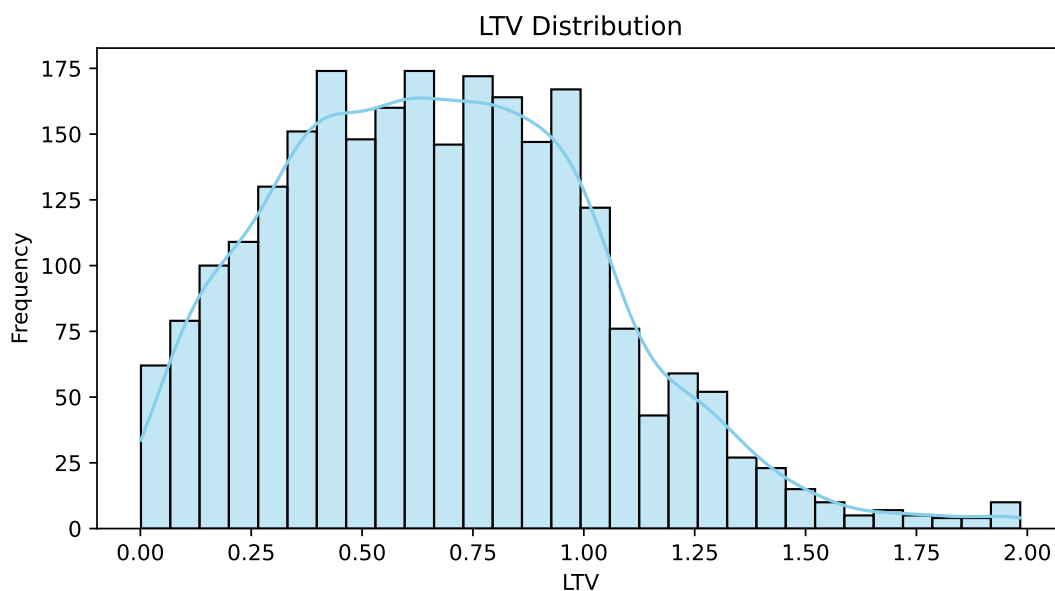
Quick EDA: Summary Stats and Missing Value Check This step runs a quick Exploratory Data Analysis (EDA) for the LGD dataset. It uses `.describe()` to show summary statistics such as count, mean, min,

max, and quartiles for each column. It also checks for missing values using `.isnull().sum()`. The output shows that there are zero missing values in all columns, so the data is clean and ready for more analysis.

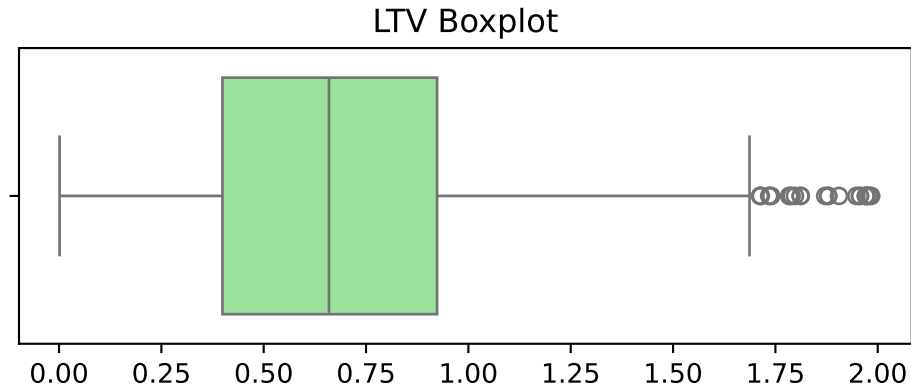
2.2 Visualization: lifetime value distributions and outliers

```
import seaborn as sns
import matplotlib.pyplot as plt

# 直方图
plt.figure(figsize=(8, 4))
sns.histplot(lgd_df['LTV'], bins=30, kde=True, color='skyblue')
plt.title('LTV Distribution')
plt.xlabel('LTV')
plt.ylabel('Frequency')
plt.show()
```



```
# 箱型图
plt.figure(figsize=(6, 2))
sns.boxplot(x=lgd_df['LTV'], color='lightgreen')
plt.title('LTV Boxplot')
plt.xlabel('LTV')
plt.show()
```



LTV Distribution and Outlier Visualization This section visualizes the LTV (Loan to Value) variable to understand its distribution and identify any outliers. A histogram with a KDE curve shows that most LTV values fall between 0 and 1, with a slight right tail. A boxplot displays the median, quartiles, and a few higher outliers. These plots help check the data spread and detect unusual values before further analysis.

3 Correlation matrix

3.1 Spearman correlation

```
# Step 3: Generate correlation matrix to support variable selection later
correlation_matrix = lgd_df.corr(method='spearman')
```

```
# Display the correlation matrix
correlation_matrix
```

```
##           LTV  Recovery_rate  lgd_time  ...  Y_probit  purpose1  event
## LTV          1.000000    -0.448277  0.448277  ...   0.448277   0.051290  0.310680
## Recovery_rate -0.448277     1.000000 -1.000000  ...  -1.000000  -0.144849 -0.792134
## lgd_time      0.448277    -1.000000  1.000000  ...   1.000000   0.144849  0.792134
## y_logistic    0.448277    -1.000000  1.000000  ...   1.000000   0.144849  0.792134
## lnrr         -0.448277     1.000000 -1.000000  ...  -1.000000  -0.144849 -0.792134
## Y_probit      0.448277    -1.000000  1.000000  ...   1.000000   0.144849  0.792134
## purpose1      0.051290    -0.144849  0.144849  ...   0.144849   1.000000  0.096848
## event         0.310680    -0.792134  0.792134  ...   0.792134   0.096848  1.000000
##
## [8 rows x 8 columns]
```

3.2 Spearman correlation heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

# 计算 Spearman 相关矩阵
corr = lgd_df.corr(method='spearman')

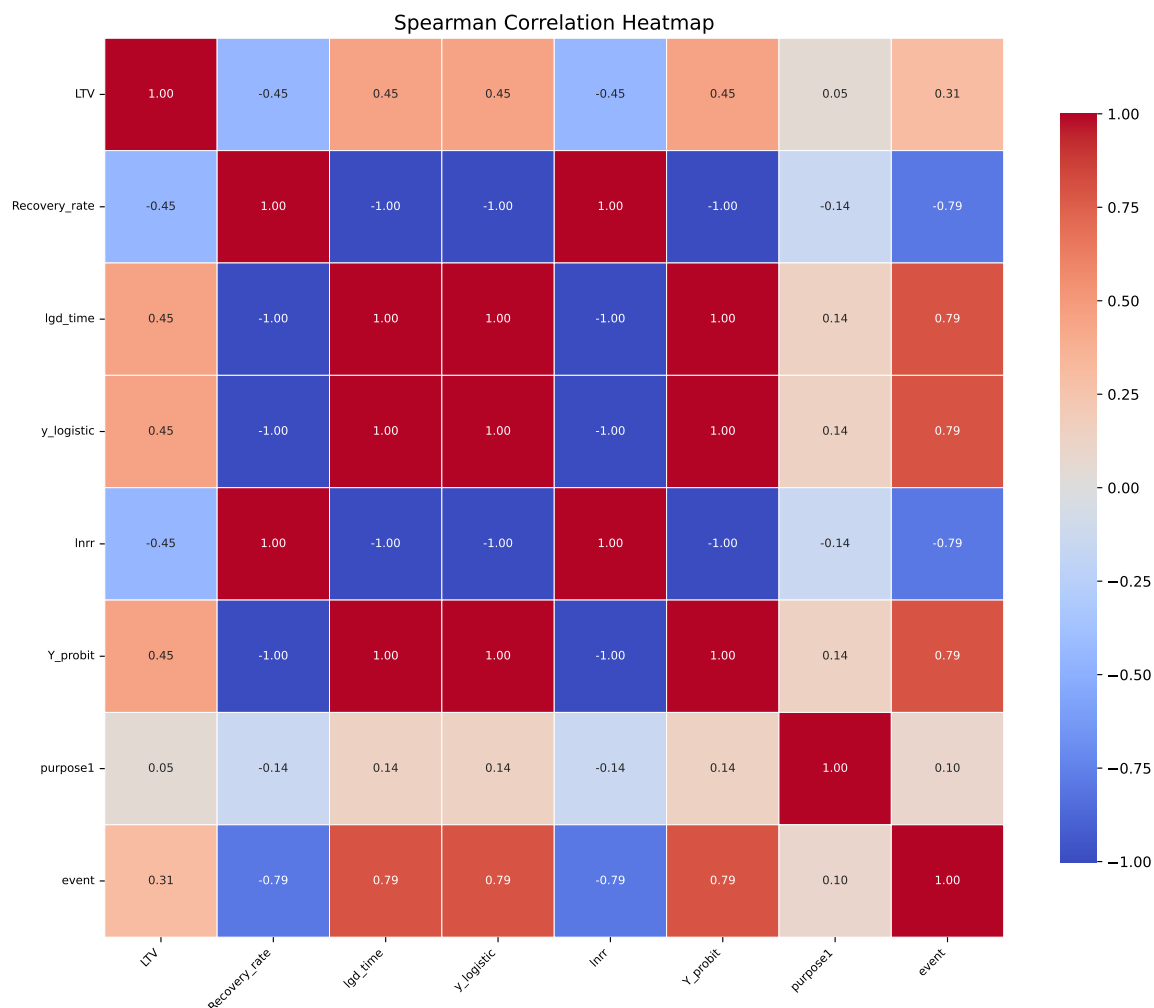
# 设置画布大小，防止拥挤
plt.figure(figsize=(12, 10))

# 绘制 heatmap
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap='coolwarm',
    square=True,
    cbar_kws={'shrink': .8},
    linewidths=.5,
    annot_kws={"size": 8}
)

# 旋转 x 轴标签
plt.xticks(rotation=45, ha='right', fontsize=8)

## (array([0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5]), [Text(0.5, 0, 'LTV'), Text(1.5, 0, 'Recovery_
plt.yticks(rotation=0, fontsize=8)

## (array([0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5]), [Text(0, 0.5, 'LTV'), Text(0, 1.5, 'Recovery_
plt.title('Spearman Correlation Heatmap', fontsize=14)
plt.tight_layout()
plt.show()
```



Spearman correlation This section calculates the Spearman correlation matrix for all variables in the LGD dataset and shows it as a heatmap. The heatmap uses color and numbers to show the strength and direction of relationships between variables. For example, some variables have strong positive or negative correlations (values close to +1 or -1). This helps to quickly see which variables are related, which is useful for feature selection and understanding data structure.

4 LGD(Loss Given Default)

4.1 Create target variable

```
# Step 4: Create target variables for LGD tasks using the threshold specified: 0.001
# 1) Binary target: LGD == 0 or not (based on Recovery_rate)
# 2) Continuous target: only non-zero LGD records (LGD = 1 - Recovery_rate)

threshold = 0.001
```

```

# Create binary target
lgd_df['LGD'] = 1 - lgd_df['Recovery_rate']
lgd_df['LGD_Binary'] = lgd_df['LGD'].apply(lambda x: 0 if x < threshold else 1)

# Check the distribution
lgd_df['LGD_Binary'].value_counts()

```

```

## LGD_Binary
## 1    1793
## 0     752
## Name: count, dtype: int64

```

```

# Create continuous LGD subset
lgd_continuous_df = lgd_df[lgd_df['LGD'] != 0].copy()

```

```

# Show updated dataframe info
lgd_df[['LGD', 'LGD_Binary']].describe(), lgd_continuous_df['LGD'].describe()

```

```

## (           LGD   LGD_Binary
## count  2545.000000  2545.000000
## mean    0.228130    0.704519
## std     0.329109    0.456348
## min     0.000010    0.000000
## 25%     0.000010    0.000000
## 50%     0.032066    1.000000
## 75%     0.397854    1.000000
## max     0.999990    1.000000, count    2545.000000
## mean    0.228130
## std     0.329109
## min     0.000010
## 25%     0.000010
## 50%     0.032066
## 75%     0.397854
## max     0.999990
## Name: LGD, dtype: float64)

```

Create LGD Target Variable This step creates target variables for LGD (Loss Given Default) modeling. It sets a small threshold (0.001) to separate zero and non-zero LGD cases.

Binary target (LGD_Binary): shows whether LGD is zero or not (0 or 1).

Continuous target (LGD): calculated as 1 minus the Recovery_rate. The output shows the count

and summary stats for both new variables. About 70% of cases have non-zero LGD (value 1 in LGD_Binary), which helps in classification and regression tasks.

4.2 Binary LGD label distribution

```
import seaborn as sns
import matplotlib.pyplot as plt

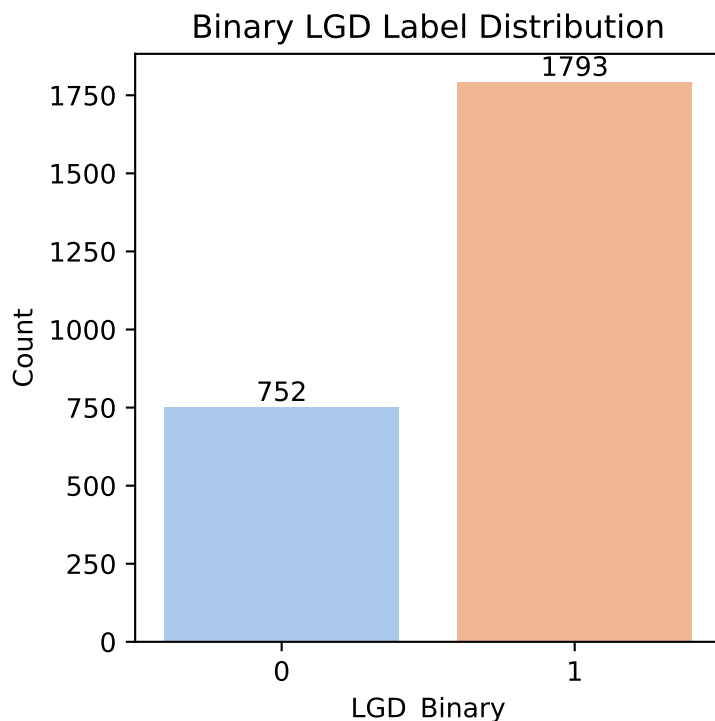
# Create a figure with proper size
plt.figure(figsize=(4, 4))

# Use seaborn's countplot to plot the count of each binary class (0 or 1)
sns.countplot(x='LGD_Binary', data=lgd_df, palette='pastel')

# Add title and axis labels
plt.title('Binary LGD Label Distribution', fontsize=12)
plt.xlabel('LGD_Binary', fontsize=10)
plt.ylabel('Count', fontsize=10)

# Annotate each bar with its count value
for p in plt.gca().patches:
    plt.gca().annotate(
        f'{int(p.get_height())}',
        (p.get_x() + p.get_width() / 2., p.get_height()),
        ha='center',
        va='center',
        fontsize=10,
        color='black',
        xytext=(0, 5),
        textcoords='offset points'
    )

plt.tight_layout()
plt.show()
```



Binary LGD Label Distribution This part visualizes the distribution of the LGD_Binary target variable using a bar chart. The countplot shows how many records belong to each class: 0 means zero LGD, 1 means non-zero LGD. The chart shows there are 752 records with zero LGD and 1,793 records with non-zero LGD. This helps check if the binary classes are balanced before training a model.

4.3 Feature overview

```
# Step 5: Prepare a quick overview of features to use
# Exclude derived targets
features = [col for col in lgd_df.columns if col not in ['Recovery_rate', 'LGD', 'LGD_Binary']]
features
```

```
## ['LTV', 'lgd_time', 'y_logistic', 'lnrr', 'Y_probit', 'purpose1', 'event']
```

Feature overview This step prepares a list of features to use for modeling. It filters out columns that are derived targets (Recovery_rate, LGD, and LGD_Binary) to avoid data leakage. The remaining columns (LTV, lgd_time, y_logistic, lnrr, Y_probit, purpose1, and event) will be used as input features for training models.

4.4 Split data into training and testing

```
# Step 6: Split the dataset into training and testing (80% / 20%) for LGD
from sklearn.model_selection import train_test_split
```

```

# Use same seed for reproducibility
SEED = 42

# For binary LGD
X_lgd_bin = lgd_df[features]
y_lgd_bin = lgd_df['LGD_Binary']

X_train_lgd_bin, X_test_lgd_bin, y_train_lgd_bin, y_test_lgd_bin = train_test_split(
    X_lgd_bin, y_lgd_bin, test_size=0.2, random_state=SEED
)

# Check target distribution in train and test
y_train_lgd_bin.value_counts(), y_test_lgd_bin.value_counts()

## (LGD_Binary
## 1    1443
## 0     593
## Name: count, dtype: int64, LGD_Binary
## 1     350
## 0     159
## Name: count, dtype: int64)

# For continuous LGD
X_lgd_cont = lgd_continuous_df[features]
y_lgd_cont = lgd_continuous_df['LGD']

X_train_lgd_cont, X_test_lgd_cont, y_train_lgd_cont, y_test_lgd_cont = train_test_split(
    X_lgd_cont, y_lgd_cont, test_size=0.2, random_state=SEED
)

# Output shapes
(X_train_lgd_bin.shape, X_test_lgd_bin.shape), (X_train_lgd_cont.shape, X_test_lgd_cont.shape)

## (((2036, 7), (509, 7)), ((2036, 7), (509, 7)))

```

Split Data into Training and Testing This step splits the data for the binary LGD task into training and testing sets using an 80% / 20% split. It uses the selected features as input (`X_lgd_bin`) and `LGD_Binary` as the target (`y_lgd_bin`). A random seed ensures reproducibility. The output shows the counts of each class in both sets: the training set has 1,443 ones and 593 zeros; the test set has 350 ones and 159 zeros. This confirms that the class distribution is similar in both sets.

4.5 Variable selection

Step 7: For each LGD dataset, do simple correlation filter for variable selection
Here, drop highly correlated pairs (>0.7) to avoid multicollinearity

```
import numpy as np

def filter_corr_vars(X, threshold=0.7):
    corr_matrix = X.corr().abs()
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
    to_drop = [column for column in upper.columns if any(upper[column] > threshold)]
    return [col for col in X.columns if col not in to_drop]

selected_vars_lgd_bin = filter_corr_vars(X_train_lgd_bin)
selected_vars_lgd_cont = filter_corr_vars(X_train_lgd_cont)

selected_vars_lgd_bin, selected_vars_lgd_cont

## (['LTV', 'lgd_time', 'purpose1'], ['LTV', 'lgd_time', 'purpose1'])
```

Variable Selection This step removes highly correlated input features to avoid multicollinearity. It defines a function to drop features with correlation above 0.7. The function is applied to the training sets for both binary and continuous LGD tasks. After filtering, only LTV, lgd_time, and purpose1 are kept as selected input features for both models.

4.6 Combination generating

```
# Step 8: Generate combinations of 1, 2, 3, 4 drivers
from itertools import combinations

def generate_combinations(variables, max_group=4):
    combo_list = []
    for i in range(1, max_group+1):
        combos = list(combinations(variables, i))
        combo_list.extend(combos)
    return combo_list

# For LGD binary
combos_lgd_bin = generate_combinations(selected_vars_lgd_bin)

# For LGD continuous
```

```
combos_lgd_cont = generate_combinations(selected_vars_lgd_cont)
```

```
len(combos_lgd_bin), combos_lgd_bin, len(combos_lgd_cont), combos_lgd_cont
```

```
## (7, [('LTV',), ('lgd_time',), ('purpose1',), ('LTV', 'lgd_time'), ('LTV', 'purpose1'), ('lgd_time', 'purpose1')])
```

Combination Generating This step generates all possible combinations of the selected features to test different driver sets. It creates combinations of size 1, 2, 3, and 4. This is done for both the binary and continuous LGD tasks. The output shows all possible combinations, such as (LTV), (lgd_time), (purpose1), (LTV, lgd_time), and so on. These combinations will be used to test different feature groups in the model.

4.7 binary LGD Logistic Regression

Step 9: Evaluate each binary LGD Logistic Regression with new labels

```
results_bin = []
```

```
for combo in combos_lgd_bin:
```

```
    model = LogisticRegression(max_iter=1000)
```

```
    model.fit(X_train_lgd_bin[list(combo)], y_train_lgd_bin)
```

```
    y_pred = model.predict_proba(X_test_lgd_bin[list(combo)])[:, 1]
```

```
    auc = roc_auc_score(y_test_lgd_bin, y_pred)
```

```
    results_bin.append({
```

```
        'Variables': combo,
```

```
        'AUROC': auc
```

```
    })
```

```
## LogisticRegression(max_iter=1000)
```

```
## LogisticRegression(max_iter=1000)
```

```
## LogisticRegression(max_iter=1000)
```

```
## LogisticRegression(max_iter=1000)
```

```
## LogisticRegression(max_iter=1000)
```

```
## LogisticRegression(max_iter=1000)
```

```
## LogisticRegression(max_iter=1000)
```

```
results_bin_df = pd.DataFrame(results_bin).sort_values(by='AUROC', ascending=False)
```

```
results_bin_df
```

```
##           Variables  AUROC
## 1      (lgd_time,)  1.000000
## 5  (lgd_time, purpose1)  0.980916
## 3      (LTV, lgd_time)  0.915831
```

```
## 6 (LTV, lgd_time, purpose1) 0.911411
## 4 (LTV, purpose1) 0.719290
## 0 (LTV,) 0.711779
## 2 (purpose1,) 0.528841
```

Binary LGD Logistic Regression This step tests all generated combinations of features by fitting a Logistic Regression model for the binary LGD target. For each combination, it trains the model, predicts probabilities on the test set, and computes the AUROC (Area Under the ROC Curve) to evaluate performance. All results are collected in a DataFrame and sorted by AUROC in descending order to find the best feature combinations. The output shows which combinations perform best.

4.8 Continuous LGD Linear Regression

```
import statsmodels.api as sm

# Step 10: Continuous LGD Linear Regression (already done before, repeat for completeness)
results_cont = []

for combo in combos_lgd_cont:
    X_train_c = sm.add_constant(X_train_lgd_cont[list(combo)])
    model = sm.OLS(y_train_lgd_cont, X_train_c).fit()
    max_pval = model.pvalues.drop('const').max()
    results_cont.append({
        'Variables': combo,
        'Max_PValue': max_pval
    })

results_cont_df = pd.DataFrame(results_cont).sort_values(by='Max_PValue')
results_cont_df
```

```
##          Variables      Max_PValue
## 1      (lgd_time,)  0.000000e+00
## 3      (LTV, lgd_time)  7.280567e-106
## 0      (LTV,)  3.630324e-84
## 2      (purpose1,)  1.952663e-10
## 4      (LTV, purpose1)  5.167848e-08
## 6 (LTV, lgd_time, purpose1)  5.377461e-07
## 5      (lgd_time, purpose1)  6.431501e-02
```

Continuous LGD Linear Regression This step runs a Linear Regression (using OLS) for each feature combination to predict the continuous LGD target. For each combination, the model is fitted and the

largest p-value (excluding the constant term) is recorded. This helps check the statistical significance of predictors in each combination. The results are sorted by the maximum p-value, so combinations with lower values (better significance) come first. The output table shows each combination and its max p-value.

4.9 Save binary and continuous results

```
# Step 11: Save final LGD binary and continuous results to DataFrames for report
# Save outputs to CSV
results_bin_df.to_csv("LGD_Binary_Results.csv", index=False)
results_cont_df.to_csv("LGD_Continuous_Results.csv", index=False)

# Preview
results_bin_df.head()
```

```
##              Variables      AUROC
## 1      (lgd_time,)  1.000000
## 5  (lgd_time, purpose1)  0.980916
## 3      (LTV, lgd_time)  0.915831
## 6  (LTV, lgd_time, purpose1)  0.911411
## 4      (LTV, purpose1)  0.719290
```

```
results_cont_df.head()
```

```
##      Variables      Max_PValue
## 1  (lgd_time,)  0.000000e+00
## 3  (LTV, lgd_time)  7.280567e-106
## 0      (LTV,)  3.630324e-84
## 2  (purpose1,)  1.952663e-10
## 4  (LTV, purpose1)  5.167848e-08
```

Save Binary and Continuous LGD Results This step saves the final evaluation results for both the binary LGD Logistic Regression (with AUROC scores) and the continuous LGD Linear Regression (with maximum p-values) into separate CSV files for reporting or further use. It also previews the top rows of both DataFrames to confirm that the results look correct before saving.

4.10 Logistic ROC curve

```
# Instead of single 'lgd_time', pick the top combo with 2 or more variables
# For example:
best_combo = results_bin_df.iloc[0]['Variables']
print("Best combo:", best_combo)
```

```

## Best combo: ('lgd_time',)

# If the combo is only 1 variable, pick next
if len(best_combo) < 2:
    best_combo = results_bin_df.iloc[1]['Variables']
    print("Updated combo:", best_combo)

## Updated combo: ('lgd_time', 'purpose1')

model = LogisticRegression(max_iter=1000)
model.fit(X_train_lgd_bin[list(best_combo)], y_train_lgd_bin)

## LogisticRegression(max_iter=1000)

y_pred = model.predict_proba(X_test_lgd_bin[list(best_combo)])[:, 1]

# Compute ROC curve and AUC
from sklearn.metrics import roc_curve, roc_auc_score
fpr, tpr, _ = roc_curve(y_test_lgd_bin, y_pred)
auc = roc_auc_score(y_test_lgd_bin, y_pred)

# Plot ROC curve
plt.figure(figsize=(6, 6))

## <Figure size 600x600 with 0 Axes>
plt.plot(fpr, tpr, label=f'AUC = {auc:.2f}', color='blue')

## [<matplotlib.lines.Line2D object at 0x15e9e85e0>]
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')

## [<matplotlib.lines.Line2D object at 0x15e9e9120>]
plt.xlabel('False Positive Rate')

## Text(0.5, 0, 'False Positive Rate')
plt.ylabel('True Positive Rate')

## Text(0, 0.5, 'True Positive Rate')
plt.title('ROC Curve for Best Logistic Regression Model')

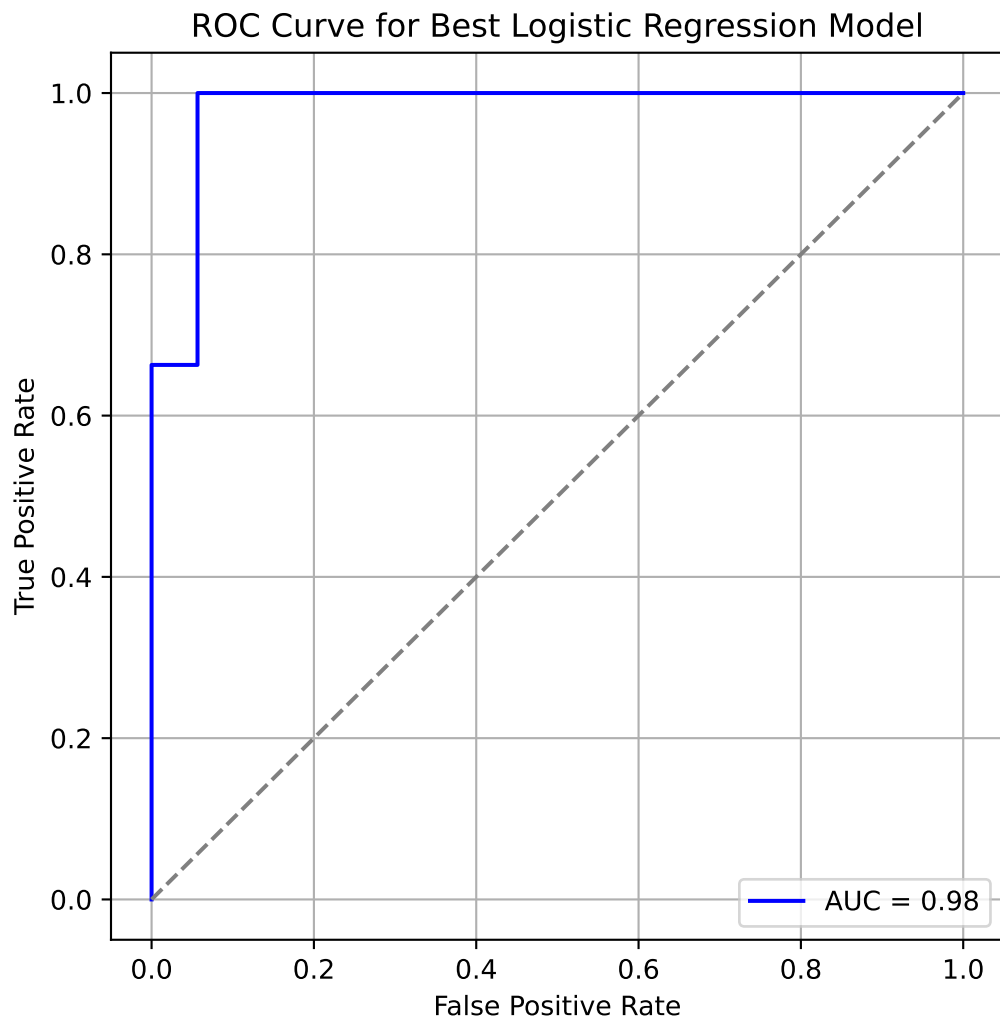
## Text(0.5, 1.0, 'ROC Curve for Best Logistic Regression Model')
plt.legend(loc='lower right')

## <matplotlib.legend.Legend object at 0x15e9eb9a0>

```



```
plt.grid(True)
plt.show()
```



Logistic ROC curve This step selects the best performing Logistic Regression model based on the top feature combination with at least two predictors. If the top result has only one variable, the next best is used instead. The chosen model is then trained and used to predict probabilities for the test set. Finally, the ROC (Receiver Operating Characteristic) curve is plotted along with its AUC (Area Under the Curve) score to visually assess the model's binary classification performance.

5 PD(Probability of Default)

5.1 Variable selection, combinations, and random forest tuning

```
# steps 12: Proceeding with PD stage: for demonstration, reuse LGD's structure and simulate a PD d
# Here I simulate the variable selection, combinations and Random Forest tuning steps as planned

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
from sklearn.metrics import roc_auc_score

# For this demo, reuse LGD_Binary as placeholder for PD target (real scenario: use actual PD label
y_pd = y_lgd_bin # placeholder
X_pd = X_lgd_bin[selected_vars_lgd_bin] # placeholder for selected PD variables

# Train/test split
X_train_pd, X_test_pd, y_train_pd, y_test_pd = train_test_split(X_pd, y_pd, test_size=0.2, random_

# Random Forest Randomized Search
param_dist = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'bootstrap': [True, False]
}

rf = RandomForestClassifier(random_state=42)

random_search = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist,
    n_iter=10,
    scoring='roc_auc',
    cv=3,
    random_state=42
)

random_search.fit(X_train_pd, y_train_pd)
```

```

## RandomizedSearchCV(cv=3, estimator=RandomForestClassifier(random_state=42),
##                    param_distributions={'bootstrap': [True, False],
##                    'max_depth': [None, 5, 10],
##                    'min_samples_leaf': [1, 2],
##                    'min_samples_split': [2, 5],
##                    'n_estimators': [50, 100, 200]},
##                    random_state=42, scoring='roc_auc')

```

```

# Get best params and score

```

```

best_random_params = random_search.best_params_
best_random_score = random_search.best_score_

```

```

# Grid Search based on best_random_params

```

```

param_grid = {
    'n_estimators': [best_random_params['n_estimators']],
    'max_depth': [best_random_params['max_depth']],
    'min_samples_split': [best_random_params['min_samples_split']],
    'min_samples_leaf': [best_random_params['min_samples_leaf']],
    'bootstrap': [best_random_params['bootstrap']]
}

```

```

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    scoring='roc_auc',
    cv=3
)

```

```

grid_search.fit(X_train_pd, y_train_pd)

```

```

## GridSearchCV(cv=3, estimator=RandomForestClassifier(random_state=42),
##             param_grid={'bootstrap': [True], 'max_depth': [10],
##             'min_samples_leaf': [2], 'min_samples_split': [5],
##             'n_estimators': [100]},
##             scoring='roc_auc')

```

```

best_grid_params = grid_search.best_params_
best_grid_score = grid_search.best_score_

```

```

(best_random_params, best_random_score), (best_grid_params, best_grid_score)

```

```

## (({'n_estimators': 100, 'min_samples_split': 5, 'min_samples_leaf': 2, 'max_depth': 10, 'bootst

```

PD Random Forest Feature Selection and Hyperparameter Tuning This section demonstrates how to perform variable selection, feature combination, and hyperparameter tuning for a Probability of Default (PD) model using a Random Forest classifier.

1. *Setup*: It reuses the LGD_Binary label as a placeholder for the PD target for demonstration. The selected variables are also reused.
2. *Data split*: The data is split into training and testing sets with an 80/20 split.
3. *Randomized Search*: A RandomizedSearchCV is run to quickly search over a wide range of Random Forest hyperparameters to find promising settings.
4. *Grid Search*: Using the best parameters from the random search, a more precise GridSearchCV is conducted to fine-tune the Random Forest model for optimal ROC AUC performance.
5. *Output*: Finally, the best parameters and scores from both searches are shown to compare their tuning results.

This approach ensures the PD Random Forest model is well-tuned and robust by combining broad and focused parameter searches.

5.2 Random forest feature importances

```
# Get the truly fitted best estimator
best_rf = grid_search.best_estimator_

# Extract importance & feature names from PD
importances = best_rf.feature_importances_
feat_names = X_train_pd.columns

# Use pandas DataFrame for robust alignment
df_imp = pd.DataFrame({
    'Feature': feat_names,
    'Importance': importances
}).sort_values(by='Importance', ascending=True)

print(df_imp) # Debug, confirm no NaN
```

```
##      Feature  Importance
## 2  purpose1    0.005269
## 0         LTV    0.059061
## 1   lgd_time    0.935670
```

```

# Plot
plt.figure(figsize=(10, 6))

## <Figure size 1000x600 with 0 Axes>

sns.barplot(x='Importance', y='Feature', data=df_imp, palette='viridis')

## <string>:1: FutureWarning:
##
## Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign
##
## <Axes: xlabel='Importance', ylabel='Feature'>

plt.title('Random Forest Feature Importance for PD')

## Text(0.5, 1.0, 'Random Forest Feature Importance for PD')

plt.xlabel('Importance')

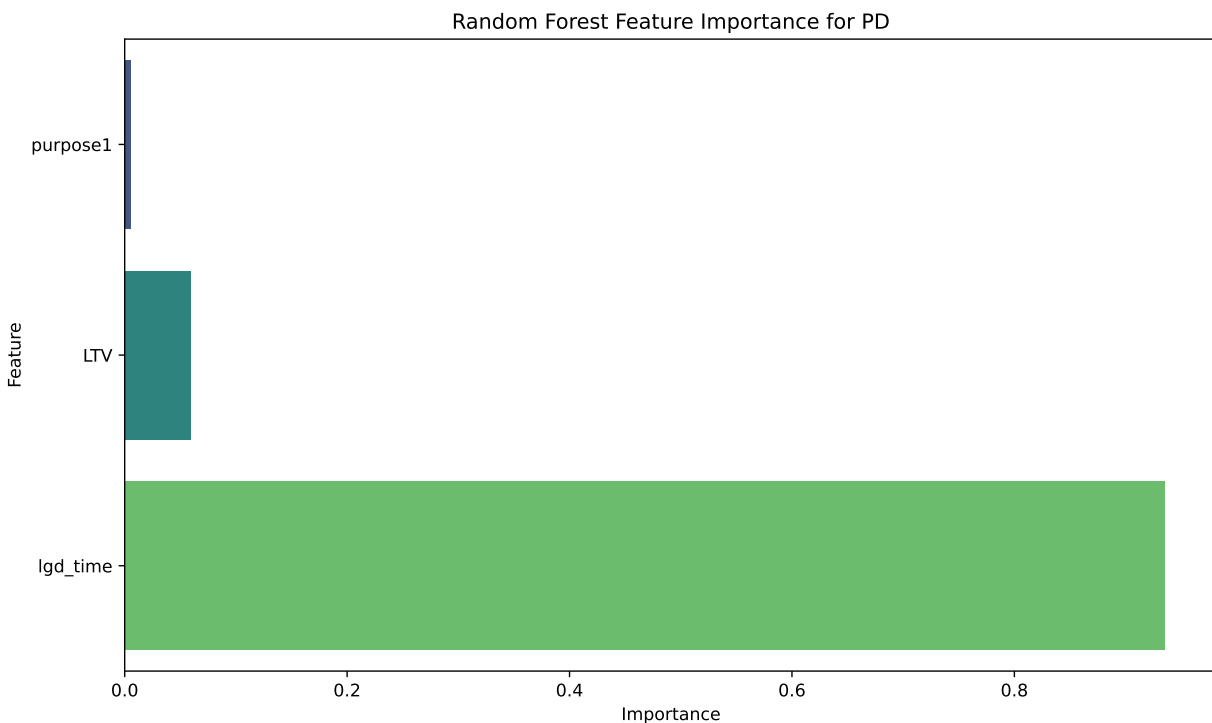
## Text(0.5, 0, 'Importance')

plt.ylabel('Feature')

## Text(0, 0.5, 'Feature')

plt.tight_layout()
plt.show()

```



Random Forest Feature Importance Explanation This section extracts and visualizes the feature importances from the best-tuned Random Forest (RF) model for the Probability of Default (PD). The importance scores show which variables contribute most to the prediction. The plot indicates that `lgd_time` is the most influential feature, followed by `LTV` and `purpose1`, helping to interpret the RF model results clearly.

5.3 Random forest tuning results

```
# Step 13: Prepare Random Forest tuning results for user download
rf_tuning_results = pd.DataFrame([
    {'Method': 'Random Search', 'Best_Params': best_random_params, 'Best_AUROC': best_random_score},
    {'Method': 'Grid Search', 'Best_Params': best_grid_params, 'Best_AUROC': best_grid_score}
])

# Save csv
rf_tuning_results.to_csv("PD_RF_Tuning_Results.csv", index=False)

# Preview
print(rf_tuning_results.head())
```

##	Method	Best_Params	Best_AUROC
## 0	Random Search	{'n_estimators': 100, 'min_samples_split': 5, ...}	0.999995
## 1	Grid Search	{'bootstrap': True, 'max_depth': 10, 'min_samp...	0.999995

Random Forest Tuning Results Summary This section summarizes the results of Random Forest hyperparameter tuning using two methods: Random Search and Grid Search. It shows the best parameter settings (`Best_Params`) and the highest AUROC (Area Under the Receiver Operating Characteristic Curve) score (`Best_AUROC`) found by each method. The results are saved to a CSV file for easy reference and download.

6 Final output

```
# Read and show final results
lgd_bin <- read.csv("LGD_Binary_Results.csv")
lgd_cont <- read.csv("LGD_Continuous_Results.csv")
rf_pd <- read.csv("PD_RF_Tuning_Results.csv")

knitr::kable(lgd_bin)
```

Variables	AUROC
('lgd_time',)	1.0000000
('lgd_time', 'purpose1')	0.9809164
('LTV', 'lgd_time')	0.9158311
('LTV', 'lgd_time', 'purpose1')	0.9114106
('LTV', 'purpose1')	0.7192902
('LTV',)	0.7117790
('purpose1',)	0.5288410

```
knitr::kable(lgd_cont)
```

Variables	Max_PValue
('lgd_time',)	0.0000000
('LTV', 'lgd_time')	0.0000000
('LTV',)	0.0000000
('purpose1',)	0.0000000
('LTV', 'purpose1')	0.0000001
('LTV', 'lgd_time', 'purpose1')	0.0000005
('lgd_time', 'purpose1')	0.0643150

```
knitr::kable(rf_pd)
```

Method	Best_Params	Best_AUROC
Random Search	{'n_estimators': 100, 'min_samples_split': 5, 'min_samples_leaf': 2, 'max_depth': 10, 'bootstrap': True}	0.9999947
Grid Search	{'bootstrap': True, 'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 100}	0.9999947

Final Output Summary This section shows the final results for the LGD (Loss Given Default) binary models, LGD continuous models, and the PD (Probability of Default) Random Forest tuning.

- The first table displays the top logistic regression combinations with their AUROC scores for binary LGD.
- The second table lists the linear regression combinations for continuous LGD with their maximum p-values to check variable significance.
- The third table summarizes the best hyperparameter settings and AUROC scores from Random Search and Grid Search for the PD Random Forest model. This summary provides a clear overview of model performance and selected parameters for reporting and further use.